National University of Computer & Emerging Sciences

Lecture 17

User and Kernel Threads

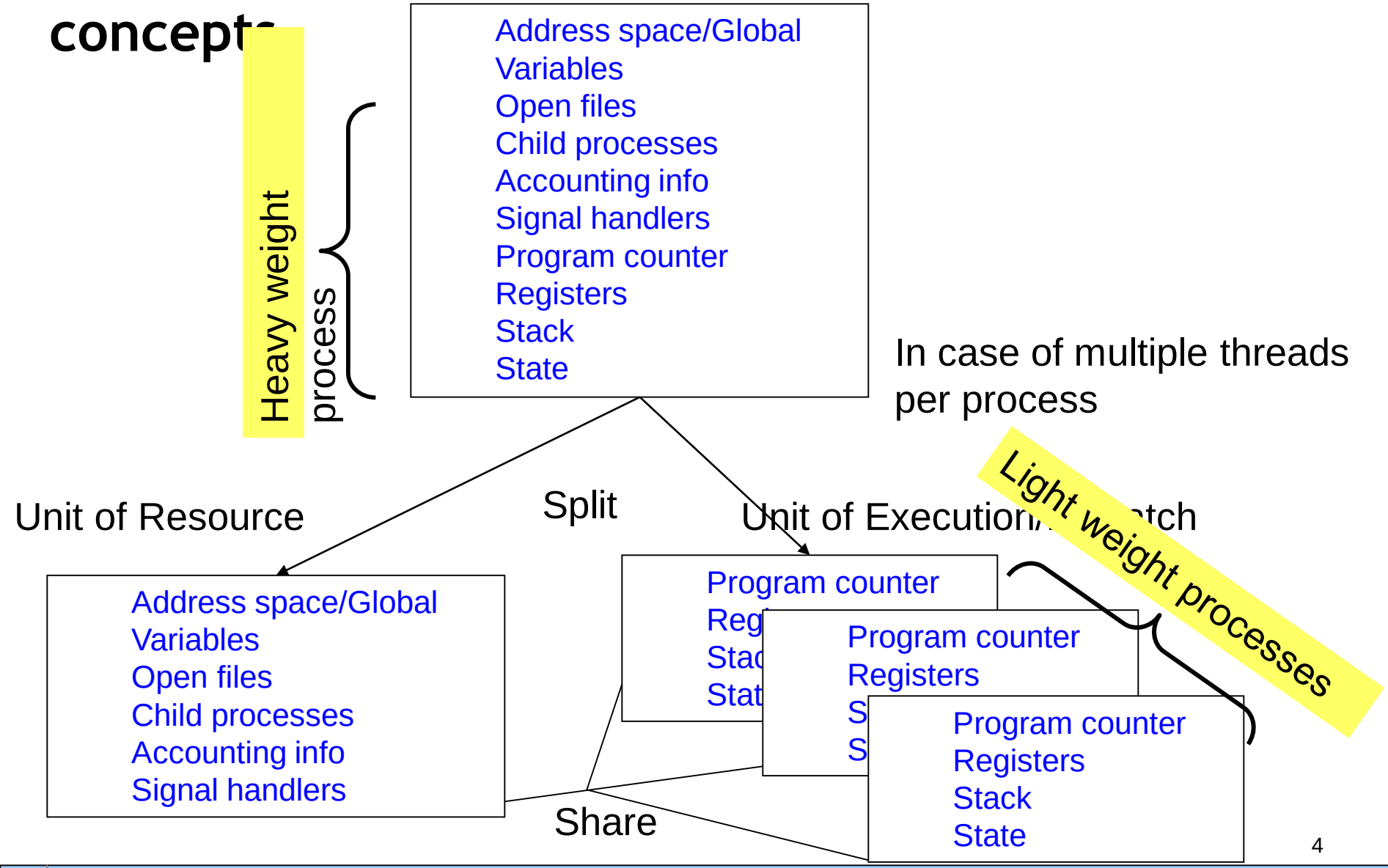
Process Characteristics

- Concept of Process has two facets.
- A Process is:
  - A Unit of resource ownership:
    - a virtual address space for the process image
    - control of some resources (files, I/O devices...)
  - A Unit of execution - process is an execution path through one or more programs
    - may be interleaved with other processes
    - execution state (Ready, Running, Blocked...) and dispatching priority

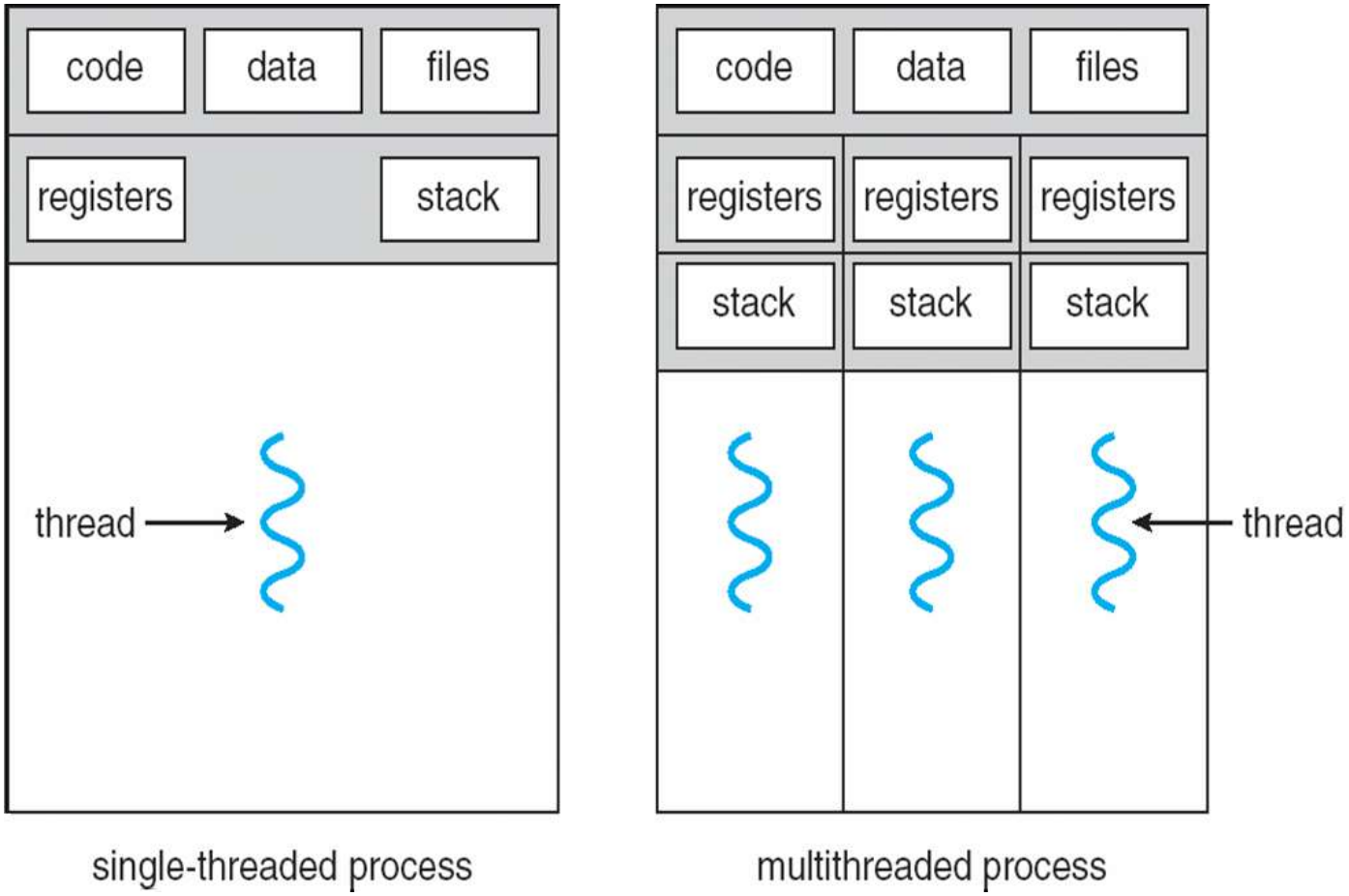
Process Characteristics

- These two characteristics are treated separately by some recent operating systems:
  - The *unit of resource ownership* is usually referred to as a *process* or *task*
  - The *unit of execution* is usually referred to a *thread* or a “lightweight process”

Its better to distinguish between the two concepts



Single and Multithreaded Processes



single-threaded process

multithreaded process

When to use threads?

- If a program can be organized into discrete, independent tasks which can execute concurrently

User Threads vs Kernel Threads

- User Thread - management done by user-level threads library
- Kernel Threads - supported by the Kernel
  - Windows XP/2000, Linux, Mac OS X…

User-Level Threads (ULT) (ex. Java)

- Kernel not aware of the existence of threads
- Thread management handled by thread library in user space
- No mode switch (kernel not involved)
- But I/O in one thread could block the entire process!

Threads library

- Contains code for:
  - creating and destroying threads
  - passing messages and data between threads
  - scheduling thread execution
    - pass control from one thread to another
  - saving and restoring thread contexts

- ULT's can be be implemented on any Operating System, because no kernel services are required to support them

Kernel Role for ULTs (None!)

- The kernel is not aware of thread activity
  - it only manages processes
- If a thread makes an I/O call, the whole process is blocked
  - Note: in the thread library that thread is still in "running" state, and will resume execution when the I/O is complete
- So thread states are independent of process states

Advantages and disadvantages of ULT

Advantages

- Thread switching does not involve the kernel: no mode switching
- Therefore fast
- Scheduling can be application specific: choose the best algorithm for the situation.
- Can run on any OS. We only need a thread library

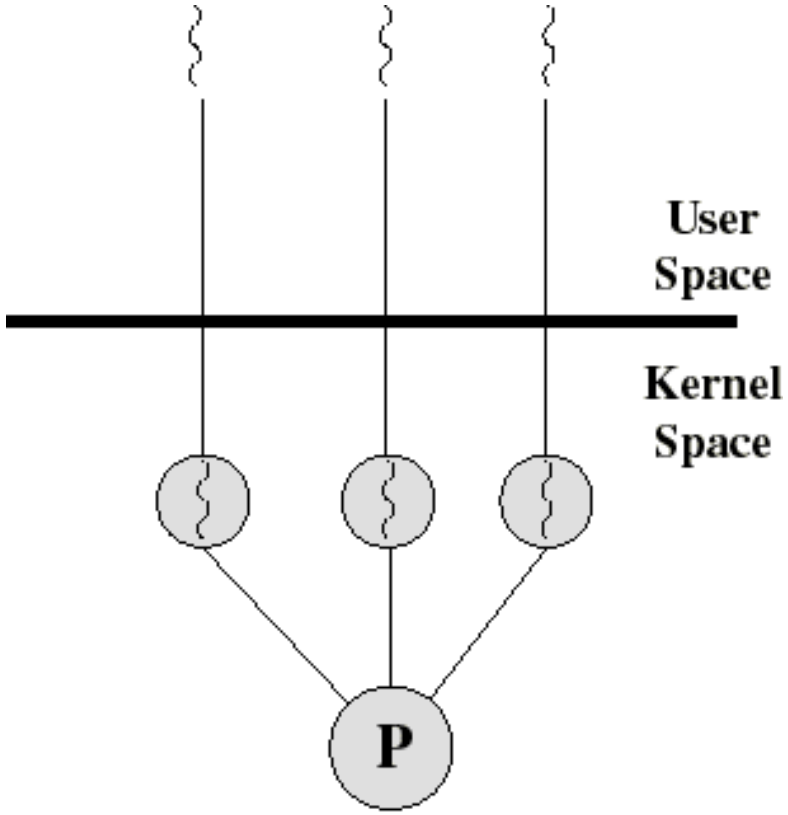
Disadvantages

- Most system calls are blocking for processes. So all threads within a process will be implicitly blocked
- The kernel can only assign processors to processes. Two threads within the same process cannot run simultaneously on two processors

Kernel-Level Threads (KLT)

Ex: Windows NT, Windows 2000, OS/2

- All thread management is done by kernel
- No thread library; instead an API to the kernel thread facility
- Kernel maintains context information for the process and the threads
- Switching between threads requires the kernel
- Kernel does Scheduling on a thread basis



Advantages and disadvantages of KLT

Advantages

- The kernel can schedule multiple threads of the same process on multiple processors
- Blocking at thread level, not process level
  - If a thread blocks, the CPU can be assigned to another thread in the same process
- Even the kernel routines can be multithreaded

Disadvantages

- Thread switching always involves the kernel. This means 2 mode switches per thread switch
- So it is slower compared to User Level Threads
  - (But faster than a full process switch)

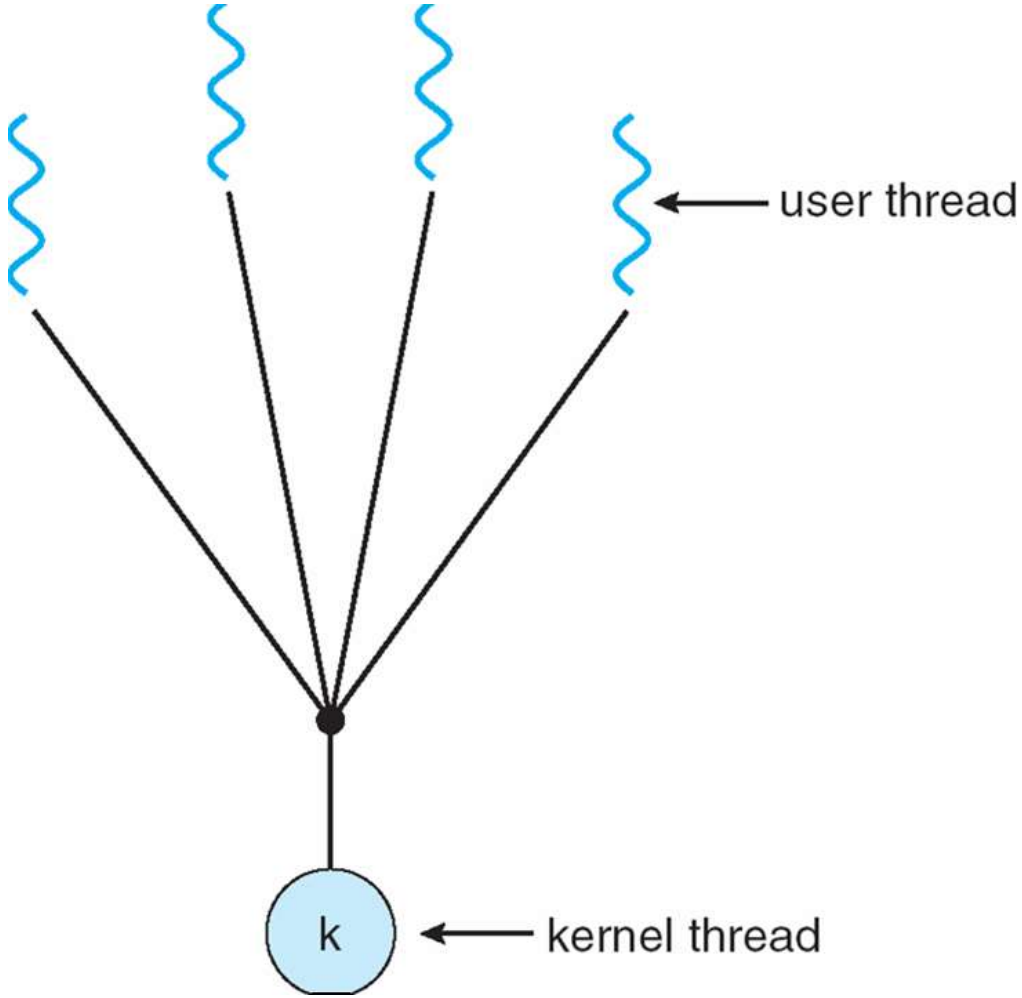
Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

Many-to-One

- Many user-level threads mapped to single kernel thread
- Examples:
  - Solaris Green Threads
  - GNU Portable Threads

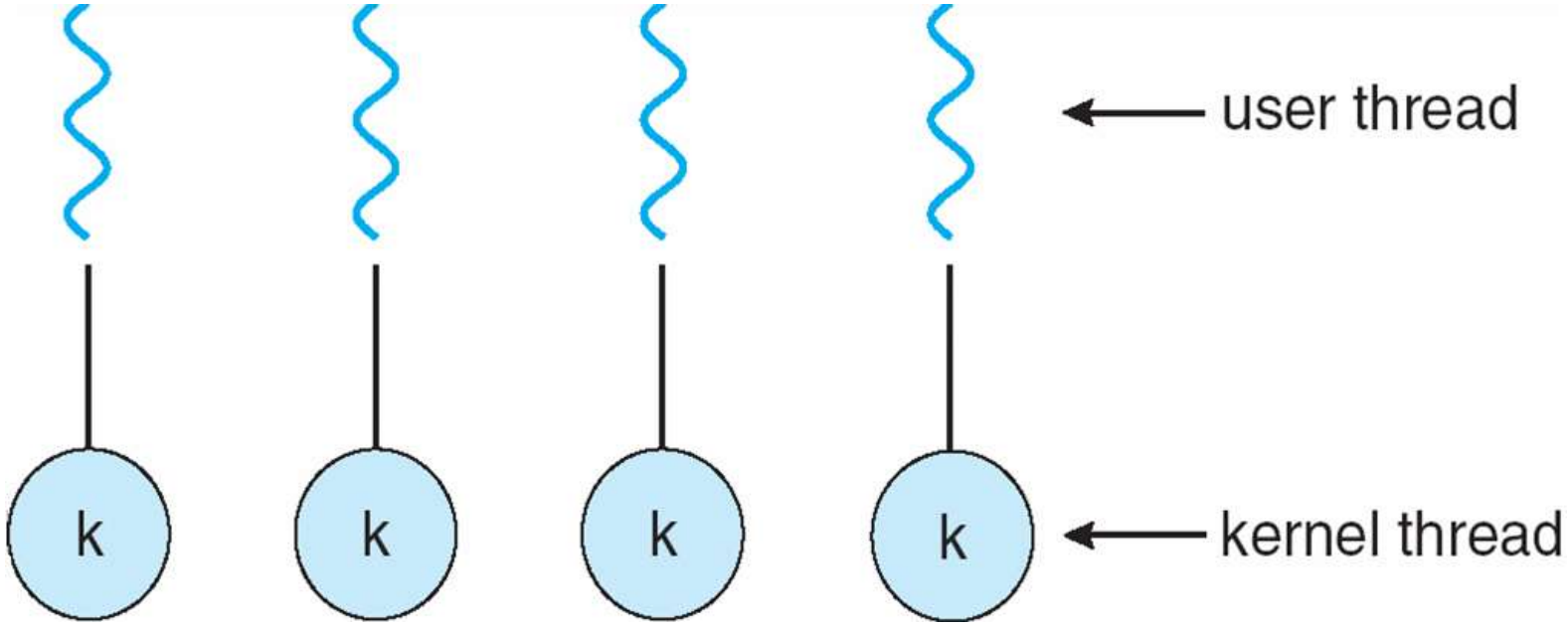
Many-to-One Model



One-to-One

- Each user-level thread maps to kernel thread
- Examples
  - Windows NT/XP/2000
  - Linux
  - Solaris 9 and later

One-to-one Model



Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows NT/2000 with the *ThreadFiber* package

Many-to-Many Model

